

Loan Status Prediction Project – Cell-by-Cell Explanation

ML Developer Explanation of the Uploaded Jupyter Notebook

Source notebook: **Loan Status Prediction Project.ipynb**

This PDF follows all 74 cells in their original order and uses the notebook's stored outputs where available.

Item	Notebook result
Total cells	74
Training rows	58,645
Features used	11 after removing id and loan_status
Target	loan_status (0 = Not Approved, 1 = Approved)
Class 0	50,295 (85.76%)
Class 1	8,350 (14.24%)
Train/Test split	75% / 25%, stratified
Test rows	14,662
Selected deployment model	Logistic Regression trained on SMOTE data (model2)
SMOTE training size	75,442 balanced rows
SMOTE test accuracy	≈ 84%
SMOTE positive-class F1	≈ 0.59
SMOTE AUC shown	≈ 0.829 (with notebook ROC-code caveat)

Project objective: predict whether a loan application will be approved while handling a strongly imbalanced target. The notebook compares class weighting, SMOTE, ADASYN, and Random Under-Sampling.

1. Complete ML Workflow

Stage	What the notebook does
1. Load data	Loads the loan training dataset.
2. Quality check	Checks shape, columns and data types.
3. Define X/Y	Removes ID and target from X; keeps loan_status as Y.
4. Analyze imbalance	Measures class counts and percentages.
5. Stratified split	Creates 75% train and 25% test sets.
6. Preprocess	Imputes, one-hot encodes categorical columns, and scales numerical columns.
7. Baseline balanced model	Logistic Regression with class_weight='balanced'.
8. SMOTE	Synthetic minority oversampling; train model2.
9. ADASYN	Adaptive synthetic oversampling; train model3.
10. Probability/ROC	Examines probabilities and ROC/AUC.
11. Deployment	Saves model2 + preprocessor and predicts new loans.
12. Under-sampling	Randomly reduces majority class and trains model4.

Scikit-learn provides accuracy, F1, confusion-matrix, ROC/AUC, and classification-report metrics for evaluating classification models. ■cite■turn0search0■turn0search1■

Cell 1 – Markdown / section explanation

Cell type: markdown

Original notebook content

Imbalanced Data

****You will observe a major difference in datapoint count related to target classes. This might introduce biased nature in model while p**

To overcome this, observe the imbalance percentage and decide what can be done:

- 1) 15-20% : we can split our data on basis of stratified split and during model building, we can pass the parameter class='balanced
- 2) Below 15%: use resampling technique.

****Resampling:****

Over sampling: creates minority classes more in such a way that the count matches with majority classes

Under sampling: Reduces majority classes more in such a way the count matches with minority classes

Business Objective: Build a model that can predict the status of Loan Approval on basis of many intermediate factors.
Data Gathering

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 2 – Load the loan dataset

Cell type: code

Original notebook content

```
import pandas as pd
df = pd.read_csv(r"https://raw.githubusercontent.com/sindhura-nk/Datasets/refs/heads/main/train_loan.csv")
df.head()
```

What this cell does

Pandas reads the training CSV from the provided dataset URL. This creates the main DataFrame `df`.

ML concept / developer understanding

The project is a supervised binary classification problem: `loan\_status` is the target.

Stored notebook output

id	person_age	person_income	person_home_ownership	person_emp_length	\
0	0	37	35000	RENT	0.0
1	1	22	56000	OWN	6.0
2	2	29	28800	OWN	8.0
3	3	30	70000	RENT	14.0
4	4	22	60000	RENT	2.0

	loan_intent	loan_grade	loan_amnt	loan_int_rate	loan_percent_income	\
0	EDUCATION	B	6000	11.49	0.17	
1	MEDICAL	C	4000	13.35	0.07	
2	PERSONAL	A	6000	8.90	0.21	
3	VENTURE	B	12000	11.11	0.17	
4	MEDICAL	A	6000	6.92	0.10	

	cb_person_default_on_file	cb_person_cred_hist_length	loan_status
0	N	14	0
1	N	2	0
2	N	10	0
3	N	5	0
4	N	3	0

Cell 3 – Markdown / section explanation

Cell type: markdown

Original notebook content

```
### Perform basic data quality check
```

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 4 – Check dataset shape

Cell type: code

Original notebook content

```
df.shape
```

What this cell does

`df.shape` shows 58,645 rows and 13 columns.

ML concept / developer understanding

Shape tells us the size of the training data before preprocessing.

Stored notebook output

```
(58645, 13)
```

Cell 5 – Inspect column names

Cell type: code

Original notebook content

```
df.columns
```

What this cell does

`df.columns` lists the 13 variables, including demographic, employment, loan, credit-history, and target columns.

ML concept / developer understanding

Knowing column names is essential for separating identifiers, features, and the target.

Stored notebook output

```
Index(['id', 'person_age', 'person_income', 'person_home_ownership',  
      'person_emp_length', 'loan_intent', 'loan_grade', 'loan_amnt',  
      'loan_int_rate', 'loan_percent_income', 'cb_person_default_on_file',  
      'cb_person_cred_hist_length', 'loan_status'],  
      dtype='object')
```

Cell 6 – Inspect structure and data types

Cell type: code

Original notebook content

```
df.info()
```

What this cell does

`df.info()` shows 58,645 non-null values in every column. Numeric columns are `int64`/`float64`; four predictor columns are categorical `object` values.

ML concept / developer understanding

This tells us which columns require numerical preprocessing and which require categorical encoding.

Stored notebook output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 58645 entries, 0 to 58644
Data columns (total 13 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   id                                    58645 non-null  int64
 1   person_age                           58645 non-null  int64
 2   person_income                        58645 non-null  int64
 3   person_home_ownership                58645 non-null  object
 4   person_emp_length                    58645 non-null  float64
 5   loan_intent                           58645 non-null  object
 6   loan_grade                           58645 non-null  object
 7   loan_amnt                            58645 non-null  int64
 8   loan_int_rate                        58645 non-null  float64
 9   loan_percent_income                  58645 non-null  float64
10   cb_person_default_on_file            58645 non-null  object
11   cb_person_cred_hist_length           58645 non-null  int64
12   loan_status                           58645 non-null  int64
dtypes: float64(3), int64(6), object(4)
memory usage: 5.8+ MB
```

Cell 7 – Markdown / section explanation

Cell type: markdown

Original notebook content

```
### Separate X and Y Features

Y : Loan Status

0-> Not Approved
1-> Approved
```

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 8 – Separate X and Y

Cell type: code

Original notebook content

```
X = df.drop(columns=['id', 'loan_status'])
Y = df[['loan_status']]

display(X.head())
print("-----")
display(Y.head())
```

What this cell does

`X` removes `id` and `loan\_status`; `Y` contains `loan\_status`.

ML concept / developer understanding

`id` is excluded as an identifier, while `loan\_status` is the prediction target: 0 = not approved, 1 = approved.

Stored notebook output

```
person_age  person_income  person_home_ownership  person_emp_length  \
0           37           35000                RENT              0.0
1           22           56000                OWN               6.0
2           29           28800                OWN              8.0
3           30           70000                RENT             14.0
4           22           60000                RENT              2.0
loan_intent  loan_grade  loan_amnt  loan_int_rate  loan_percent_income  \
```

0	EDUCATION	B	6000	11.49	0.17
1	MEDICAL	C	4000	13.35	0.07
2	PERSONAL	A	6000	8.90	0.21
3	VENTURE	B	12000	11.11	0.17
4	MEDICAL	A	6000	6.92	0.10

	cb_person_default_on_file	cb_person_cred_hist_length
0	N	14
1	N	2
2	N	10
3	N	5
4	N	3

	loan_status
0	0
1	0
2	0
3	0
4	0

Cell 9 – Markdown / section explanation

Cell type: markdown

Original notebook content

Observe whether Y data is balanced or not.

Meaning: DO we have moderate datapoints for Class approved and Not approved(0,1)

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 10 – Check target class counts

Cell type: code

Original notebook content

```
Y['loan_status'].value_counts()
```

What this cell does

The target contains 50,295 class-0 records and 8,350 class-1 records.

ML concept / developer understanding

This is a major class imbalance: the approved class is the minority.

Stored notebook output

```
loan_status
0      50295
1       8350
Name: count, dtype: int64
```

Cell 11 – Calculate class percentages

Cell type: code

Original notebook content

```
((Y['loan_status'].value_counts())/(len(df)))*100
```

What this cell does

Class 0 represents about 85.76% and class 1 about 14.24% of the dataset.

ML concept / developer understanding

A model can obtain high accuracy by favoring the majority class, so imbalance-aware evaluation is important.

Stored notebook output

```
loan_status
0      85.761787
1     14.238213
Name: count, dtype: float64
```

Cell 12 – Visualize class imbalance

Cell type: code

Original notebook content

```
Y['loan_status'].value_counts().plot(kind='barh')
```

What this cell does

A horizontal bar chart displays the counts for the two loan-status classes.

ML concept / developer understanding

Visualization makes the imbalance immediately visible.

Stored notebook output

```
<Axes: ylabel='loan_status'>
<Figure size 640x480 with 1 Axes>
```

Cell 13 – Markdown / section explanation

Cell type: markdown

Original notebook content

```
### The dataset is imbalanced in nature

### Technique 1: splitting the data on basis of stratified split
```

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 14 – Stratified train/test split

Cell type: code

Original notebook content

```
from sklearn.model_selection import train_test_split
xtrain1,xtest1,ytrain1,ytest1 = train_test_split(X,Y,train_size=0.75,stratify=Y)
```

What this cell does

`train\_test\_split(..., train\_size=0.75, stratify=Y)` creates a 75% training and 25% testing split while preserving class proportions.

ML concept / developer understanding

Stratification is important for imbalanced classification because both sets should retain similar target proportions.

Cell 15 – Check training feature shape

Cell type: code

Original notebook content

```
xtrain1.shape
```

What this cell does

The training feature matrix has 43,983 rows and 11 columns.

ML concept / developer understanding

This confirms 75% of the 58,645 records were used for training and 11 predictors remain after dropping ID and target.

Stored notebook output

```
(43983, 11)
```

Cell 16 – Check training target shape

Cell type: code

Original notebook content

```
ytrain1.shape
```

What this cell does

`ytrain1` contains 43,983 target values.

ML concept / developer understanding

The number of target rows matches the training feature rows.

Stored notebook output

```
(43983, 1)
```

Cell 17 – Inspect training features

Cell type: code

Original notebook content

```
xtrain1.head()
```

What this cell does

Displays sample training records before preprocessing.

ML concept / developer understanding

This helps verify that categorical and numerical variables are present as expected.

Stored notebook output

person_age	person_income	person_home_ownership	person_emp_length	\
36843	30	54000	MORTGAGE	5.0
19248	22	60000	RENT	6.0
269	23	65000	RENT	2.0
54883	45	49800	MORTGAGE	13.0
31611	23	33600	RENT	4.0

	loan_intent	loan_grade	loan_amnt	loan_int_rate	\
36843	DEBTCONSOLIDATION	B	10000	9.99	
19248	EDUCATION	B	12000	10.62	
269	EDUCATION	C	8000	12.68	
54883	HOMEIMPROVEMENT	A	2400	6.99	
31611	EDUCATION	C	5000	14.26	

	loan_percent_income	cb_person_default_on_file	\
36843	0.19	N	
19248	0.20	N	
269	0.12	N	
54883	0.05	N	
31611	0.15	N	

cb_person_cred_hist_length

36843	5
19248	3
269	3
54883	12
31611	3

Cell 18 – Inspect testing features

Cell type: code

Original notebook content

```
xtest1.head()
```

What this cell does

Displays sample test records before preprocessing.

ML concept / developer understanding

The test set is kept separate from model fitting.

Stored notebook output

person_age	person_income	person_home_ownership	person_emp_length	\
37767	23	50000	MORTGAGE	7.0
44069	28	80000	MORTGAGE	12.0
33720	25	130000	MORTGAGE	10.0
30446	34	30000	MORTGAGE	5.0
43671	24	32640	MORTGAGE	8.0

	loan_intent	loan_grade	loan_amnt	loan_int_rate	\
37767	EDUCATION	A	6000	8.49	
44069	VENTURE	B	12000	11.71	
33720	MEDICAL	B	24250	12.53	
30446	MEDICAL	A	3000	7.51	
43671	HOMEIMPROVEMENT	A	3000	7.51	

	loan_percent_income	cb_person_default_on_file	\
37767	0.12	N	
44069	0.15	N	
33720	0.19	N	
30446	0.10	N	
43671	0.09	N	

	cb_person_cred_hist_length
37767	3
44069	7
33720	2
30446	6
43671	2

Cell 19 – Markdown / section explanation

Cell type: markdown

Original notebook content

```
### Data Preprocessing and Data Cleaning
```

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 20 – Build preprocessing pipeline

Cell type: code

Original notebook content

```

from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer

cat = list(X.select_dtypes(include='object').columns)
con = list(X.select_dtypes(include='number').columns)

num_pipe = make_pipeline(SimpleImputer(strategy='median'),
                        StandardScaler())

cat_pipe = make_pipeline(SimpleImputer(strategy='most_frequent'),
                        OneHotEncoder(handle_unknown='ignore', sparse_output=False))

pre = ColumnTransformer([('cat', cat_pipe, cat),
                        ('con', num_pipe, con)]).set_output(transform='pandas')

pre

```

What this cell does

The notebook detects categorical and numerical columns. Categorical columns use most-frequent imputation plus `OneHotEncoder(handle\_unknown='ignore')`; numerical columns use median imputation plus `StandardScaler`. `ColumnTransformer` combines both pipelines.

ML concept / developer understanding

This is a strong preprocessing design: categorical variables become numeric dummy columns, while numerical variables are imputed and standardized. `handle\_unknown='ignore'` helps inference when a new category appears.

Stored notebook output

```

ColumnTransformer(transformers=[('cat',
                                Pipeline(steps=[('simpleimputer',
                                                  SimpleImputer(strategy='most_frequent')),
                                                  ('onehotencoder',
                                                  OneHotEncoder(handle_unknown='ignore',
                                                                sparse_output=False))]),
                                ['person_home_ownership', 'loan_intent',
                                'loan_grade', 'cb_person_default_on_file']),
                              ('con',
                                Pipeline(steps=[('simpleimputer',
                                                  SimpleImputer(strategy='median')),
                                                  ('standardscaler',
                                                  StandardScaler())]),
                                ['person_age', 'person_income',
                                'person_emp_length', 'loan_amnt',
                                'loan_int_rate', 'loan_percent_income',
                                'cb_person_cred_hist_length'])])

```

Cell 21 – Fit preprocessing on training data

Cell type: code

Original notebook content

```
pre.fit(xtrain1)
```

What this cell does

`pre.fit(xtrain1)` learns imputation, encoding categories, and scaling parameters using training data only.

ML concept / developer understanding

Fitting preprocessing only on training data prevents test-set information leakage.

Stored notebook output

```

ColumnTransformer(transformers=[('cat',
                                Pipeline(steps=[('simpleimputer',
                                                  SimpleImputer(strategy='most_frequent')),
                                                  ('onehotencoder',
                                                  OneHotEncoder(handle_unknown='ignore',
                                                                sparse_output=False))]),
                                ['person_home_ownership', 'loan_intent',
                                'loan_grade', 'cb_person_default_on_file']),
                              ('con',
                                Pipeline(steps=[('simpleimputer',

```

```

        SimpleImputer(strategy='median')),
        ('standardscaler',
         StandardScaler()))],
['person_age', 'person_income',
 'person_emp_length', 'loan_amnt',
 'loan_int_rate', 'loan_percent_income',
 'cb_person_cred_hist_length']])

```

Cell 22 – Transform train and test data

Cell type: code

Original notebook content

```

xtrain1_pre = pre.transform(xtrain1)
xtest1_pre = pre.transform(xtest1)

```

What this cell does

The fitted preprocessor transforms both datasets into model-ready pandas DataFrames.

ML concept / developer understanding

The same learned transformations must be applied to future/test data.

Cell 23 – Inspect transformed features

Cell type: code

Original notebook content

```
xtrain1_pre.head(2)
```

What this cell does

The output shows one-hot encoded categorical columns such as `cat\_\_person\_home\_ownership\_RENT` and scaled numerical features.

ML concept / developer understanding

After preprocessing, the model receives numerical features only.

Stored notebook output

```

cat__person_home_ownership_MORTGAGE  cat__person_home_ownership_OTHER  \
36843                                1.0                                0.0
19248                                0.0                                0.0

cat__person_home_ownership_OWN  cat__person_home_ownership_RENT  \
36843                          0.0                          0.0
19248                          0.0                          1.0

cat__loan_intent_DEBTCONSOLIDATION  cat__loan_intent_EDUCATION  \
36843                              1.0                              0.0
19248                              0.0                              1.0

cat__loan_intent_HOMEIMPROVEMENT  cat__loan_intent_MEDICAL  \
36843                             0.0                             0.0
19248                             0.0                             0.0

cat__loan_intent_PERSONAL  cat__loan_intent_VENTURE  ...  \
36843                    0.0                    0.0  ...
19248                    0.0                    0.0  ...

cat__loan_grade_G  cat__cb_person_default_on_file_N  \
36843             0.0                               1.0
19248             0.0                               1.0

cat__cb_person_default_on_file_Y  con__person_age  con__person_income  \
36843                          0.0          0.404422          -0.266494
19248                          0.0          -0.918586          -0.104733

con__person_emp_length  con__loan_amnt  con__loan_int_rate  \
36843          0.078881          0.141594          -0.224705
19248          0.333202          0.500674          -0.016983

con__loan_percent_income  con__cb_person_cred_hist_length

```

36843	0.335308	-0.201495
19248	0.444301	-0.697512

[2 rows x 26 columns]

Cell 24 – Markdown / section explanation

Cell type: markdown

Original notebook content

Model Building

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 25 – Train Logistic Regression with class balancing

Cell type: code

Original notebook content

```
from sklearn.linear_model import LogisticRegression
model1 = LogisticRegression(class_weight='balanced')
model1.fit(xtrain1_pre, ytrain1)
```

What this cell does

`LogisticRegression(class\_weight='balanced')` trains a classifier that automatically gives greater weight to the minority class. The notebook records a `DataConversionWarning` because `Y` is a one-column `DataFrame` instead of a 1D target.

ML concept / developer understanding

The purpose is to reduce the effect of class imbalance without creating synthetic samples.

Stored notebook output

```
e:\DATA SCIENTIST\ML\repository\venv\lib\site-packages\sklearn\utils\validation.py:1406: DataConversionWarning: A column-vector y was passed as a 1D target. This will cause DataConversionWarning: A column-vector y was passed as a 1D target.
```

```
LogisticRegression(class_weight='balanced')
```

Cell 26 – Training accuracy

Cell type: code

Original notebook content

```
model1.score(xtrain1_pre, ytrain1)
```

What this cell does

Training accuracy is about 84.44%.

ML concept / developer understanding

Accuracy alone is not enough for this imbalanced dataset, so the notebook checks F1 and class-level metrics next.

Stored notebook output

```
0.8443716890616829
```

Cell 27 – Testing accuracy

Cell type: code

Original notebook content

```
model1.score(xtest1_pre, ytest1)
```

What this cell does

Testing accuracy is about 83.94%.

ML concept / developer understanding

The train/test scores are close, suggesting no large train-test accuracy gap in this experiment.

Stored notebook output

```
0.8394489155640431
```

Cell 28 – Markdown / section explanation

Cell type: markdown

Original notebook content

```
### Performance Metrics
```

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 29 – Predict and calculate F1

Cell type: code

Original notebook content

```
from sklearn.metrics import f1_score, ConfusionMatrixDisplay, classification_report

ypreds = model1.predict(xtest1_pre)

f1_data = f1_score(ytest1, ypreds)
f1_data
```

What this cell does

The model predicts the test set and the notebook calculates $F1 \approx 0.5902$.

ML concept / developer understanding

F1 balances precision and recall for the positive class; it is more informative than accuracy when the minority class matters.

Stored notebook output

```
0.5901810584958217
```

Cell 30 – Display confusion matrix

Cell type: code

Original notebook content

```
ConfusionMatrixDisplay.from_estimator(model1, xtest1_pre, ytest1)
```

What this cell does

Creates a confusion matrix for model1 on the test set.

ML concept / developer understanding

The matrix shows how many approved and non-approved applications are correctly or incorrectly classified.

Stored notebook output

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x2289854ba60>  
<Figure size 640x480 with 2 Axes>
```

Cell 31 – Classification report

Cell type: code

Original notebook content

```
print(classification_report(ytest1,ypreds))
```

What this cell does

The report shows class 0 F1 $\approx$ 0.90 and class 1 F1 $\approx$ 0.59, with overall accuracy $\approx$ 0.84.

ML concept / developer understanding

The minority class has much weaker precision than the majority class, even though recall is relatively high.

Stored notebook output

	precision	recall	f1-score	support
0	0.96	0.84	0.90	12574
1	0.46	0.81	0.59	2088
accuracy			0.84	14662
macro avg	0.71	0.83	0.75	14662
weighted avg	0.89	0.84	0.86	14662

Cell 32 – Markdown / section explanation

Cell type: markdown

Original notebook content

```
### Technique3: Resampling Method
```

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 33 – Install imbalanced-learn

Cell type: code

Original notebook content

```
pip install imblearn
```

What this cell does

Runs `pip install imblearn`; the environment already has `imbalanced-learn` installed.

ML concept / developer understanding

The package provides resampling methods such as SMOTE, ADASYN, and RandomUnderSampler.

Stored notebook output

```
Requirement already satisfied: imblearn in e:\data scientist\ml\repository\venv\lib\site-packages (0.0)  
Requirement already satisfied: imbalanced-learn in e:\data scientist\ml\repository\venv\lib\site-packages (from imblearn) (0.14.2)
```

Requirement already satisfied: numpy<3,>=1.25.2 in e:\data scientist\ml\repository\venv\lib\site-packages (from imbalanced-learn->imblea
Requirement already satisfied: scipy<2,>=1.11.4 in e:\data scientist\ml\repository\venv\lib\site-packages (from imbalanced-learn->imblea
Requirement already satisfied: scikit-learn<2,>=1.4.2 in e:\data scientist\ml\repository\venv\lib\site-packages (from imbalanced-learn->imblea
Requirement already satisfied: sklearn-compat<0.2,>=0.1.6 in e:\data scientist\ml\repository\venv\lib\site-packages (from imbalanced-learn->imblea
Requirement already satisfied: joblib<2,>=1.2.0 in e:\data scientist\ml\repository\venv\lib\site-packages (from imbalanced-learn->imblea
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in e:\data scientist\ml\repository\venv\lib\site-packages (from imbalanced-learn->imblea
Requirement already satisfied: cloudpickle>=3.0 in e:\data scientist\ml\repository\venv\lib\site-packages (from joblib<2,>=1.2.0->imbal
Note: you may need to restart the kernel to use updated packages.

Cell 34 – Markdown / section explanation

Cell type: markdown

Original notebook content

```
### Restart the kernel and run all the cells
```

Resampling:

1. Over sampling -> SMOTE , ADASYN SMOTE: Synthetic Minority Over sampling technique ADASYN : Adaptive Synthetic sample technique
2. Under sampling

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 35 – Check test-set size

Cell type: code

Original notebook content

```
len(xtest1_pre)
```

What this cell does

The test set contains 14,662 rows.

ML concept / developer understanding

The test set remains unchanged while resampling is applied only to training data.

Stored notebook output

```
14662
```

Cell 36 – Check training imbalance

Cell type: code

Original notebook content

```
ytrain1['loan_status'].value_counts()
```

What this cell does

The training target has 37,721 class-0 and 6,262 class-1 observations.

ML concept / developer understanding

This confirms the same imbalance exists in the training partition.

Stored notebook output

```
loan_status
0      37721
1       6262
Name: count, dtype: int64
```

Cell 37 – Apply SMOTE

Cell type: code

Original notebook content

```
from imblearn.over_sampling import SMOTE, ADASYN

smote = SMOTE()

x_saml, y_saml = smote.fit_resample(xtrain1_pre, ytrain1)
```

What this cell does

SMOTE creates synthetic minority-class samples until the training classes are balanced.

ML concept / developer understanding

SMOTE is an oversampling method that creates synthetic minority examples rather than simply duplicating existing rows.

Cell 38 – Check resampled feature count

Cell type: code

Original notebook content

```
len(x_saml)
```

What this cell does

SMOTE produces 75,442 training rows.

ML concept / developer understanding

The total doubles because both classes now have 37,721 observations.

Stored notebook output

```
75442
```

Cell 39 – Check resampled target length

Cell type: code

Original notebook content

```
len(y_saml)
```

What this cell does

The resampled target also contains 75,442 rows.

ML concept / developer understanding

X and y remain aligned after resampling.

Stored notebook output

```
75442
```

Cell 40 – Verify SMOTE balance

Cell type: code

Original notebook content

```
y_saml['loan_status'].value_counts()
```

What this cell does

Both classes have exactly 37,721 observations.

Cell 44 – SMOTE confusion matrix

Cell type: code

Original notebook content

```
ConfusionMatrixDisplay.from_estimator(model2, xtest1_pre, ytest1)
```

What this cell does

Creates a confusion matrix for model2 on the original untouched test set.

ML concept / developer understanding

Evaluation remains fair because the test set was not SMOTE-resampled.

Stored notebook output

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x228b52a6c50>  
<Figure size 640x480 with 2 Axes>
```

Cell 45 – SMOTE classification report

Cell type: code

Original notebook content

```
ypred_sampl = model2.predict(xtest1_pre)  
print(classification_report(ytest1, ypred_sampl))
```

What this cell does

The test report gives class-1 precision ≈ 0.47 , recall ≈ 0.81 , F1 ≈ 0.59 and overall accuracy ≈ 0.84 .

ML concept / developer understanding

SMOTE keeps minority recall high, but precision remains limited.

Stored notebook output

	precision	recall	f1-score	support
0	0.96	0.85	0.90	12574
1	0.47	0.81	0.59	2088
accuracy			0.84	14662
macro avg	0.72	0.83	0.75	14662
weighted avg	0.89	0.84	0.86	14662

Cell 46 – Apply ADASYN

Cell type: code

Original notebook content

```
ada = ADASYN()  
x_sampl_ada, y_sampl_ada = ada.fit_resample(xtrain1_pre, ytrain1)  
y_sampl_ada['loan_status'].value_counts()
```

What this cell does

ADASYN creates a synthetic balanced-ish training set, producing 37,721 class-0 and 37,675 class-1 samples.

ML concept / developer understanding

ADASYN focuses synthetic generation more on difficult minority examples.

Stored notebook output

```
loan_status  
0    37721  
1    37675  
Name: count, dtype: int64
```

Cell 47 – Train Logistic Regression on ADASYN

Cell type: code

Original notebook content

```
model3 = LogisticRegression()  
model3.fit(x_sampl_ada, y_sampl_ada)
```

What this cell does

`model3` is trained using the ADASYN-resampled training data.

ML concept / developer understanding

The test set remains untouched.

Stored notebook output

```
e:\DATA SCIENTIST\ML\repository\venv\lib\site-packages\sklearn\utils\validation.py:1406: DataConversionWarning: A column-vector y was passed as a 1D array, which will be deprecated in the future. Use y = column_or_1d(y, warn=True) instead.  
y = column_or_1d(y, warn=True)  
  
LogisticRegression()
```

Cell 48 – ADASYN training accuracy

Cell type: code

Original notebook content

```
model3.score(x_sampl_ada, y_sampl_ada)
```

What this cell does

Training accuracy on the ADASYN data is about 75.31%.

ML concept / developer understanding

This is lower than the training scores of the other approaches.

Stored notebook output

```
0.7530638230144835
```

Cell 49 – ADASYN confusion matrix

Cell type: code

Original notebook content

```
ConfusionMatrixDisplay.from_estimator(model3, xtest1_pre, ytest1)
```

What this cell does

Displays model3's confusion matrix on the original test data.

ML concept / developer understanding

This visualizes the trade-off between catching approved loans and avoiding false approvals.

Stored notebook output

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x228b60c5690>  
<Figure size 640x480 with 2 Axes>
```

Cell 50 – ADASYN classification report

Cell type: code

Original notebook content

```
ypreds_ada = model3.predict(xtest1_pre)  
print(classification_report(ytest1, ypreds_ada))
```

What this cell does

Test accuracy is about 0.77; class-1 recall is 0.87 but precision is only 0.37 and F1 is 0.52.

ML concept / developer understanding

ADASYN increases minority recall but creates many false positives in this notebook's experiment.

Stored notebook output

	precision	recall	f1-score	support	
0		0.97	0.76	0.85	12574
1		0.37	0.87	0.52	2088
accuracy				0.77	14662
macro avg		0.67	0.81	0.69	14662
weighted avg		0.89	0.77	0.80	14662

Cell 51 – Markdown / section explanation

Cell type: markdown

Original notebook content

```
### Logistic regression generates results in probabilities
```

```
Default threshold: 0.5
```

```
Category the data into classes on basis of the above threshold. (>0.5: Class A, <0.5: Class B)
```

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 52 – Inspect prediction probabilities

Cell type: code

Original notebook content

```
probs = model3.predict_proba(xtest1_pre)
print(probs[:5])
print(ypreds_ada[:5])
```

What this cell does

`predict\_proba` returns the probability assigned to classes 0 and 1. The first five predictions are all class 0.

ML concept / developer understanding

Logistic Regression produces probabilities before the final class decision.

Stored notebook output

```
[[0.82473316 0.17526684]
 [0.86676197 0.13323803]
 [0.81993486 0.18006514]
 [0.75403772 0.24596228]
 [0.67864975 0.32135025]]
[0 0 0 0]
```

Cell 53 – Markdown / section explanation

Cell type: markdown

Original notebook content

```
### ROC Metric: ROC Curve
Receiver Operator Characterstic
```

Analysis: If the Area under curve(AUC) is more than 0.80(80%), the model is resulting in accurate predictions

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 54 – Calculate ROC/AUC for SMOTE model

Cell type: code

Original notebook content

```
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt
ypred = model2.predict_proba(xtest1_pre)[: ,1] # probabilities for positive class
fpr, tpr, thresholds = roc_curve(ytest1, ypred_sampl)
roc_auc_score = auc(fpr, tpr) # Main auc for each model
print(roc_auc_score)
```

What this cell does

The notebook obtains positive-class probabilities with `predict\_proba` but then passes `ypred\_sampl` (hard class predictions) to `roc\_curve`. It calculates AUC ≈ 0.8286 .

ML concept / developer understanding

ML Developer note: ROC curves should normally use continuous scores/probabilities such as `ypred`, not hard 0/1 predictions. The notebook's code should be corrected if this metric is used for model selection.

Stored notebook output

0.8286220669422459

Cell 55 – Plot ROC curve

Cell type: code

Original notebook content

```
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt
ypred = model2.predict_proba(xtest1_pre)[: ,1]
fpr, tpr, thresholds = roc_curve(ytest1, ypred_sampl)
roc_auc_score = auc(fpr, tpr)
# # auc_scores = roc_auc_score
plt.plot (fpr, tpr, label=f'{model2} AUC={roc_auc_score:.2f}')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f'ROC and AUC curve for {model2}')
plt.legend()
plt.show()
```

What this cell does

Plots the FPR and TPR calculated in the previous cell and labels the curve with model2's AUC.

ML concept / developer understanding

The chart visualizes the ROC result produced by the notebook's calculation.

Stored notebook output

<Figure size 640x480 with 1 Axes>

Cell 56 – Markdown / section explanation

Cell type: markdown

Original notebook content

```
### We can finalise model2 created using SMOTE technique for Deployment.  
  
### Save the pipeline and finalised model
```

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 57 – Save final model and preprocessing

Cell type: code

Original notebook content

```
import joblib  
joblib.dump(model2, "Model_SMOTE.joblib")  
joblib.dump(pre, "Preprocessor.joblib")
```

What this cell does

Saves `model2` as `Model\_SMOTE.joblib` and the preprocessing transformer as `Preprocessor.joblib`.

ML concept / developer understanding

Saving both is essential because new data must undergo the exact same preprocessing before prediction.

Stored notebook output

```
['Preprocessor.joblib']
```

Cell 58 – Markdown / section explanation

Cell type: markdown

Original notebook content

```
### Out of sample predictions
```

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 59 – Load new loan applications

Cell type: code

Original notebook content

```
xnew = pd.read_csv(r"https://raw.githubusercontent.com/sindhura-nk/Datasets/refs/heads/main/test_loan.csv")  
xnew.head()
```

What this cell does

Reads the test loan CSV and displays the first five new applications.

ML concept / developer understanding

These records do not contain the target and are used for out-of-sample prediction.

Stored notebook output

id	person_age	person_income	person_home_ownership	person_emp_length	\
0	58645	23	69000	RENT	3.0
1	58646	26	96000	MORTGAGE	6.0
2	58647	26	30000	RENT	5.0
3	58648	33	50000	RENT	4.0
4	58649	26	102000	MORTGAGE	8.0

	loan_intent	loan_grade	loan_amnt	loan_int_rate	\
0	HOMEIMPROVEMENT	F	25000	15.76	
1	PERSONAL	C	10000	12.68	
2	VENTURE	E	4000	17.19	
3	DEBTCONSOLIDATION	A	7000	8.90	
4	HOMEIMPROVEMENT	D	15000	16.32	

	loan_percent_income	cb_person_default_on_file	cb_person_cred_hist_length
0	0.36	N	2
1	0.10	Y	4
2	0.13	Y	2
3	0.14	N	7
4	0.15	Y	4

Cell 60 – Markdown / section explanation

Cell type: markdown

Original notebook content

```
### Load the saved preprocessor and model
```

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 61 – Load saved artifacts

Cell type: code

Original notebook content

```
pre_loaded = joblib.load("Preprocessor.joblib")
model_loaded = joblib.load("Model_SMOTE.joblib")
```

What this cell does

Loads the saved preprocessing transformer and SMOTE-trained Logistic Regression model from disk.

ML concept / developer understanding

This simulates a deployment workflow where training is already complete.

Cell 62 – Preprocess new data

Cell type: code

Original notebook content

```
xnew_pre = pre_loaded.transform(xnew)
xnew_pre.head()
```

What this cell does

Applies the saved preprocessor to the new loan records.

ML concept / developer understanding

The model receives the same encoded/scaled feature structure used during training.

Stored notebook output

```
cat__person_home_ownership_MORTGAGE  cat__person_home_ownership_OTHER  \
0                                     0.0                               0.0
1                                     1.0                               0.0
2                                     0.0                               0.0
3                                     0.0                               0.0
4                                     1.0                               0.0

cat__person_home_ownership_OWN  cat__person_home_ownership_RENT  \
0                               0.0                               1.0
1                               0.0                               0.0
2                               0.0                               1.0
3                               0.0                               1.0
4                               0.0                               0.0

cat__loan_intent_DEBTCONSOLIDATION  cat__loan_intent_EDUCATION  \
0                                   0.0                               0.0
1                                   0.0                               0.0
2                                   0.0                               0.0
3                                   1.0                               0.0
4                                   0.0                               0.0

cat__loan_intent_HOMEIMPROVEMENT  cat__loan_intent_MEDICAL  \
0                                  1.0                               0.0
1                                  0.0                               0.0
2                                  0.0                               0.0
3                                  0.0                               0.0
4                                  1.0                               0.0

cat__loan_intent_PERSONAL  cat__loan_intent_VENTURE  ...  \
0                          0.0                       0.0  ...
1                          1.0                       0.0  ...
2                          0.0                       1.0  ...
3                          0.0                       0.0  ...
4                          0.0                       0.0  ...

cat__loan_grade_G  cat__cb_person_default_on_file_N  \
0                  0.0                               1.0
1                  0.0                               0.0
2                  0.0                               0.0
3                  0.0                               1.0
4                  0.0                               0.0

cat__cb_person_default_on_file_Y  con__person_age  con__person_income  \
0                                0.0              -0.753210             0.137908
1                                1.0
... [long stored output shortened]
```

Cell 63 – Predict loan status

Cell type: code

Original notebook content

```
final_preds = model_loaded.predict(xnew_pre)
final_preds[:10]
```

What this cell does

The saved model predicts the first ten new applications as `[1,0,1,0,1,1,0,0,1,0]`.

ML concept / developer understanding

1 means approved and 0 means not approved according to the notebook's target definition.

Stored notebook output

```
array([1, 0, 1, 0, 1, 1, 0, 0, 1, 0])
```

Cell 64 – Attach predictions to original data

Cell type: code

Original notebook content

```
xnew['Loan Status Approval Prediction'] = final_preds
xnew.head()
```


What this cell does

Adds a `Loan Status Approval Prediction` column to the new applications.

ML concept / developer understanding

This makes the prediction output readable and business-friendly.

Stored notebook output

id	person_age	person_income	person_home_ownership	person_emp_length	\
0	58645	23	69000	RENT	3.0
1	58646	26	96000	MORTGAGE	6.0
2	58647	26	30000	RENT	5.0
3	58648	33	50000	RENT	4.0
4	58649	26	102000	MORTGAGE	8.0

	loan_intent	loan_grade	loan_amnt	loan_int_rate	\
0	HOMEIMPROVEMENT	F	25000	15.76	
1	PERSONAL	C	10000	12.68	
2	VENTURE	E	4000	17.19	
3	DEBTCONSOLIDATION	A	7000	8.90	
4	HOMEIMPROVEMENT	D	15000	16.32	

	loan_percent_income	cb_person_default_on_file	cb_person_cred_hist_length	\
0	0.36	N		2
1	0.10	Y		4
2	0.13	Y		2
3	0.14	N		7
4	0.15	Y		4

	Loan Status Approval Prediction
0	1
1	0
2	1
3	0
4	1

Cell 65 – Export prediction results

Cell type: code

Original notebook content

```
xnew.to_csv("Predicted results.csv",index=False)
```

What this cell does

Saves the final prediction table to `Predicted results.csv`.

ML concept / developer understanding

The CSV can be shared with another application, analyst, or reporting workflow.

Cell 66 – Markdown / section explanation

Cell type: markdown

Original notebook content

```
### Random Under Sampler --> use it only for Large datasets
```

What this cell does

This cell provides the notebook's section title, instructions, interpretation, or conceptual explanation. It does not execute Python code.

ML concept / developer understanding

The markdown is kept as part of the original learning flow.

Cell 67 – Random Under-Sampling

Cell type: code

Original notebook content

```
from imblearn.under_sampling import RandomUnderSampler
rand = RandomUnderSampler()
x_sampl_under, y_sampl_under = rand.fit_resample(xtrain1_pre, ytrain1)
y_sampl_under['loan_status'].value_counts()
```

What this cell does

RandomUnderSampler reduces the majority class until both classes contain 6,262 training samples.

ML concept / developer understanding

Under-sampling is useful when datasets are large, but it discards majority-class observations.

Stored notebook output

```
loan_status
0      6262
1      6262
Name: count, dtype: int64
```

Cell 68 – Train Logistic Regression after under-sampling

Cell type: code

Original notebook content

```
model4 = LogisticRegression()
model4.fit(x_sampl_under, y_sampl_under)
```

What this cell does

Trains `model4` on the balanced under-sampled data.

ML concept / developer understanding

The model learns from equal class counts but with substantially fewer training examples.

Stored notebook output

```
e:\DATA SCIENTIST\ML\repository\venv\lib\site-packages\sklearn\utils\validation.py:1406: DataConversionWarning: A column-vector y was passed
  y = column_or_1d(y, warn=True)

LogisticRegression()
```

Cell 69 – Under-sampling training accuracy

Cell type: code

Original notebook content

```
model4.score(x_sampl_under, y_sampl_under)
```

What this cell does

Training accuracy is about 83.40%.

ML concept / developer understanding

This is measured on the reduced training set.

Stored notebook output

```
0.8339987224528904
```

Cell 70 – Under-sampling confusion matrix

Cell type: code

Original notebook content

```
ConfusionMatrixDisplay.from_estimator(model4, xtest1_pre, ytest1)
```

What this cell does

Displays model4's confusion matrix on the original test set.

ML concept / developer understanding

Again, evaluation is performed on untouched test data.

Stored notebook output

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x228b5713e20>  
<Figure size 640x480 with 2 Axes>
```

Cell 71 – Under-sampling classification report

Cell type: code

Original notebook content

```
ypred_under= model4.predict(xtest1_pre)  
print(classification_report(ytest1,ypred_under))
```

What this cell does

Test accuracy is about 0.84; class-1 precision $\approx$ 0.47, recall $\approx$ 0.81, F1 $\approx$ 0.60.

ML concept / developer understanding

This experiment produces the highest positive-class F1 among the three reported resampling approaches in the notebook, though it is essentially tied with SMOTE.

Stored notebook output

	precision	recall	f1-score	support	
	0	0.96	0.85	0.90	12574
	1	0.47	0.81	0.60	2088
accuracy				0.84	14662
macro avg		0.72	0.83	0.75	14662
weighted avg		0.89	0.84	0.86	14662

Cell 72 – Calculate under-sampling AUC

Cell type: code

Original notebook content

```
fpr1,tpr1,thresholds = roc_curve(ytest1,ypred_under)  
  
roc_auc_score2 = auc(fpr1,tpr1)  
print(roc_auc_score2)
```

What this cell does

Calculates an AUC of about 0.8293 using `ypred\_under` as the score input.

ML concept / developer understanding

ML Developer note: as with cell 54, hard class predictions are used rather than continuous probabilities. A more appropriate ROC calculation would use `model4.predict\_proba(xtest1\_pre)[:,1]`.

Stored notebook output

```
0.829260928559632
```

Cell 73 – Plot under-sampling ROC curve

Cell type: code

Original notebook content

```
plt.plot(fpr1,tpr1,label=f"{model4}: Score: {roc_auc_score2:.2f}")
```

```
plt.xlabel("False Positive rate")
plt.ylabel("True Positive rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```

What this cell does

Plots the FPR/TPR values and labels the chart with the AUC.

ML concept / developer understanding

The chart visualizes the notebook's under-sampling ROC calculation.

Stored notebook output

<Figure size 640x480 with 1 Axes>

Cell 74 – Empty cell

Cell type: code

What this cell does

No code or output is present.

ML concept / developer understanding

No additional operation occurs.

2. Class Imbalance – The Main Problem

The target is heavily imbalanced: 85.76% class 0 versus 14.24% class 1. This means a model that mostly predicts class 0 can look accurate while performing poorly on loan approvals. That is why the notebook compares several imbalance-handling techniques.

The notebook uses four approaches:

- **Class weighting:** model1 uses `class_weight='balanced'`.
- **SMOTE:** creates synthetic minority samples; model2.
- **ADASYN:** generates synthetic samples with more focus on difficult cases; model3.
- **Random Under-Sampling:** removes majority samples; model4.

3. Model Comparison From the Notebook

Approach	Test Accuracy	Class-1 Precision	Class-1 Recall	Class-1 F1
Class weight (model1)	83.94%	0.46	0.81	0.59
SMOTE (model2)	≈84%	0.47	0.81	0.59
ADASYN (model3)	≈77%	0.37	0.87	0.52
Under-sampling (model4)	≈84%	0.47	0.81	0.60

Based strictly on the notebook's reported test metrics, model4 has the highest positive-class F1 (0.60), while model2 is the model the notebook explicitly finalizes for deployment. This distinction is important: the notebook's chosen deployment model is model2, even though model4's reported class-1 F1 is slightly higher.

4. Important ML Developer Notes

1. **DataConversionWarning:** several LogisticRegression cells pass a one-column DataFrame target. A cleaner target shape is 1D, for example `ytrain1['loan_status']` or `.ravel()`. The warning does not mean training failed.
2. **ROC/AUC code issue:** in cells 54 and 72, the notebook calculates ``predict_proba(...)[:,1]`` into ``ypred``, but then passes hard predictions (``ypred_sample` / `ypred_under``) into ``roc_curve``. For a standard ROC curve, use the continuous positive-class probabilities. Scikit-learn's ROC-AUC interface expects prediction scores/probabilities rather than hard class labels. [■cite■turn0search5■](#)
3. **Test-set discipline:** the resampling methods are applied to the training data, while the original test set is used for evaluation. That is the correct general pattern for evaluating resampling methods.
4. **Deployment:** saving both ``Model_SMOTE.joblib`` and ``Preprocessor.joblib`` is important because the deployed model needs the same encoding and scaling learned during training.
5. **Business interpretation:** for loan approval, false positives and false negatives may have different costs. Therefore, accuracy alone should not decide the production model; precision, recall, F1, ROC-AUC, and business risk should all be considered.

5. Interview-Ready Questions & Answers

Why is this dataset imbalanced?

Because 85.76% of applications are class 0 and only 14.24% are class 1 in the notebook.

Why use stratified train/test split?

To preserve approximately the same class proportions in both training and testing sets.

Why use OneHotEncoder?

Several input features such as home ownership, loan intent, loan grade, and default history are categorical. One-hot encoding converts their categories into numerical columns.

Why use StandardScaler?

It standardizes numerical features so their scales are comparable and helps numerical optimization.

Why use SMOTE?

To synthetically increase minority-class training examples and balance the training data.

What is ADASYN?

An adaptive oversampling technique that focuses more synthetic generation around difficult minority examples.

What is Random Under-Sampling?

It reduces the majority class by randomly removing majority examples until the classes are balanced.

Why not evaluate on resampled test data?

Because the test set should represent the original real-world distribution. Resampling it can make evaluation less representative.

Which model did the notebook deploy?

The notebook explicitly finalizes ``model2``, the Logistic Regression trained after SMOTE.

What does class 1 mean?

According to the notebook, class 1 means Loan Approved.

What was the positive-class F1 of model2?

Approximately 0.59 on the original test set.

What is the key improvement you would make?

Correct the ROC/AUC calculation to use positive-class probabilities, and compare models using an appropriate validation strategy and business cost analysis.

6. Final Project Understanding

You built a complete imbalanced binary-classification pipeline: **load loan data** → **identify imbalance** → **stratified split** → **numerical/categorical preprocessing** → **Logistic Regression** → **class weighting** → **SMOTE** → **ADASYN** → **probability analysis** → **ROC/AUC** → **save model and preprocessor** → **predict new loan applications** → **Random Under-Sampling comparison**.